BITS WILP / NSP1

EvalSphere

Benchmark Smarter. Decide Faster. · Assignment 2

01 GITHUB REPOSITORY

Complete source code · requirements.txt · README.md · test CSV data

https://github.com/sreeram348/2026ak05608_assignment_2.git

02 LIVE STREAMLIT COMMUNITY CLOUD APP

https://2026ak05608assignment2git-tgcyacyp3j2ftfuwllshbx.streamlit.app/

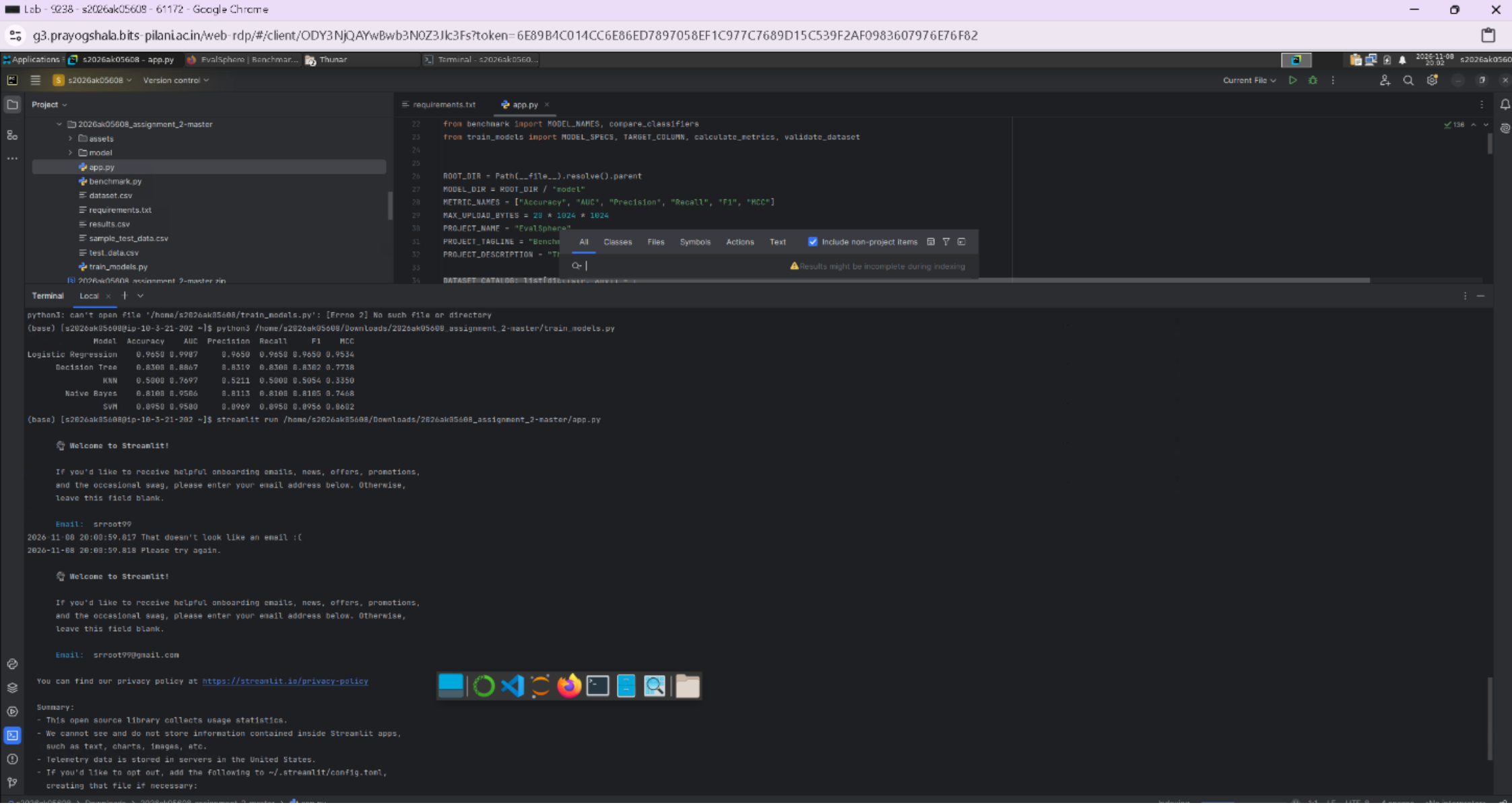
03 LAB EXECUTION & APPLICATION SCREENSHOTS

Four screenshots follow: model training results, the running Streamlit server, the assignment workspace, and the live app open in the lab browser.

Created by SRIRAM POLAMARASETTI

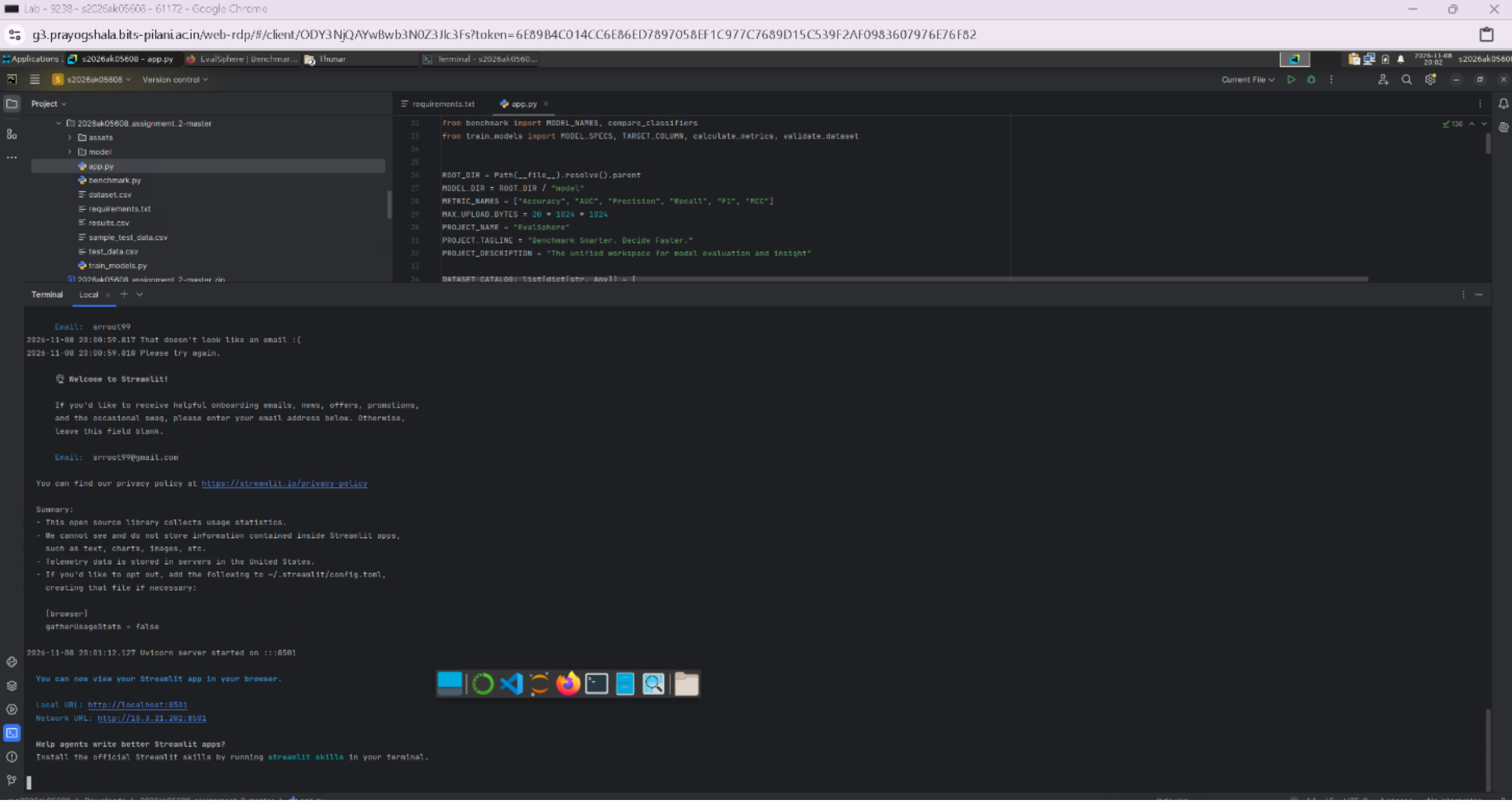
BITS ID 2026ak05608

Project details and user instructions follow the screenshots.



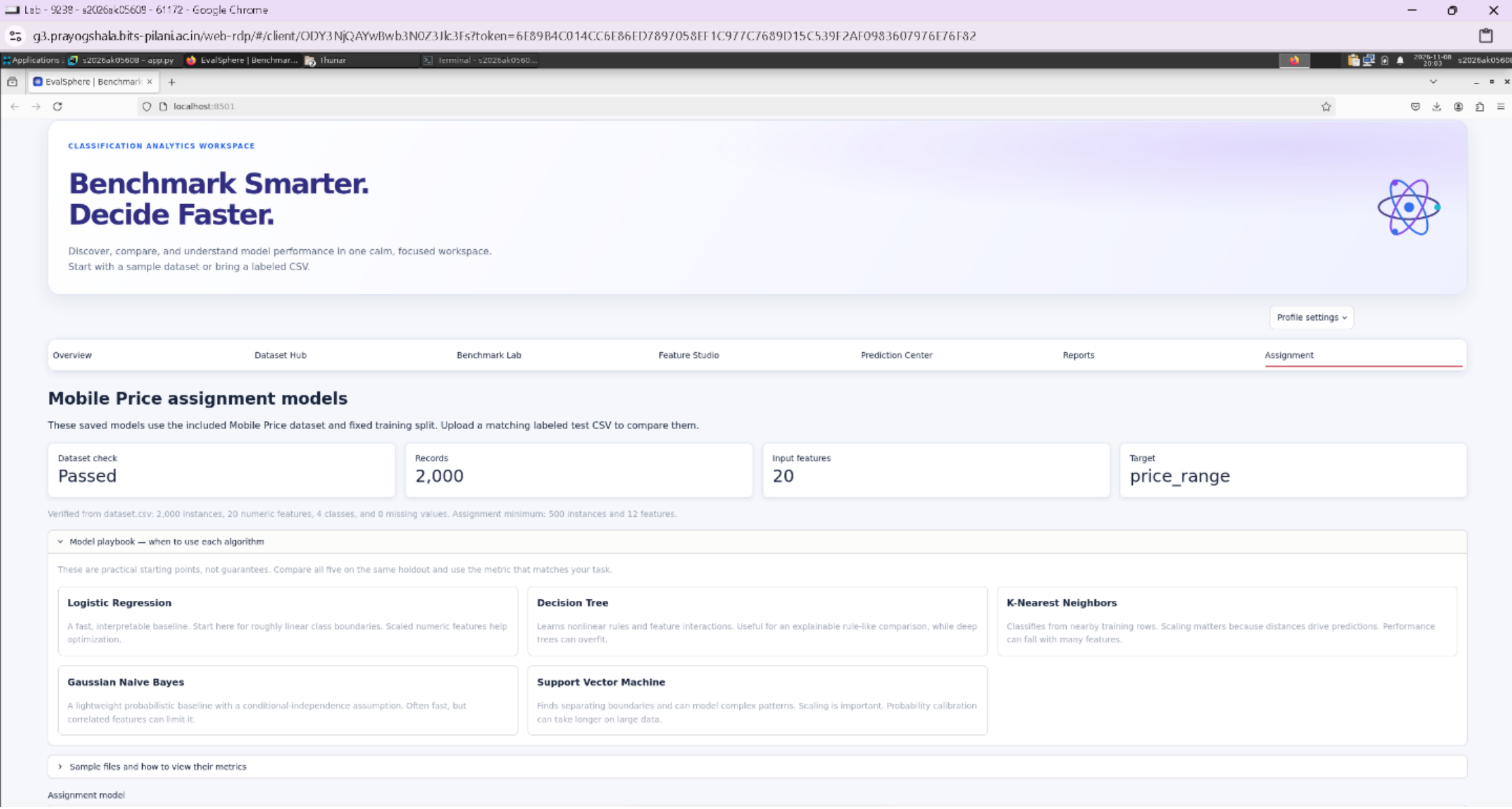
01 Model training and benchmark results

The lab workspace with the project code and the five-classifier evaluation output.



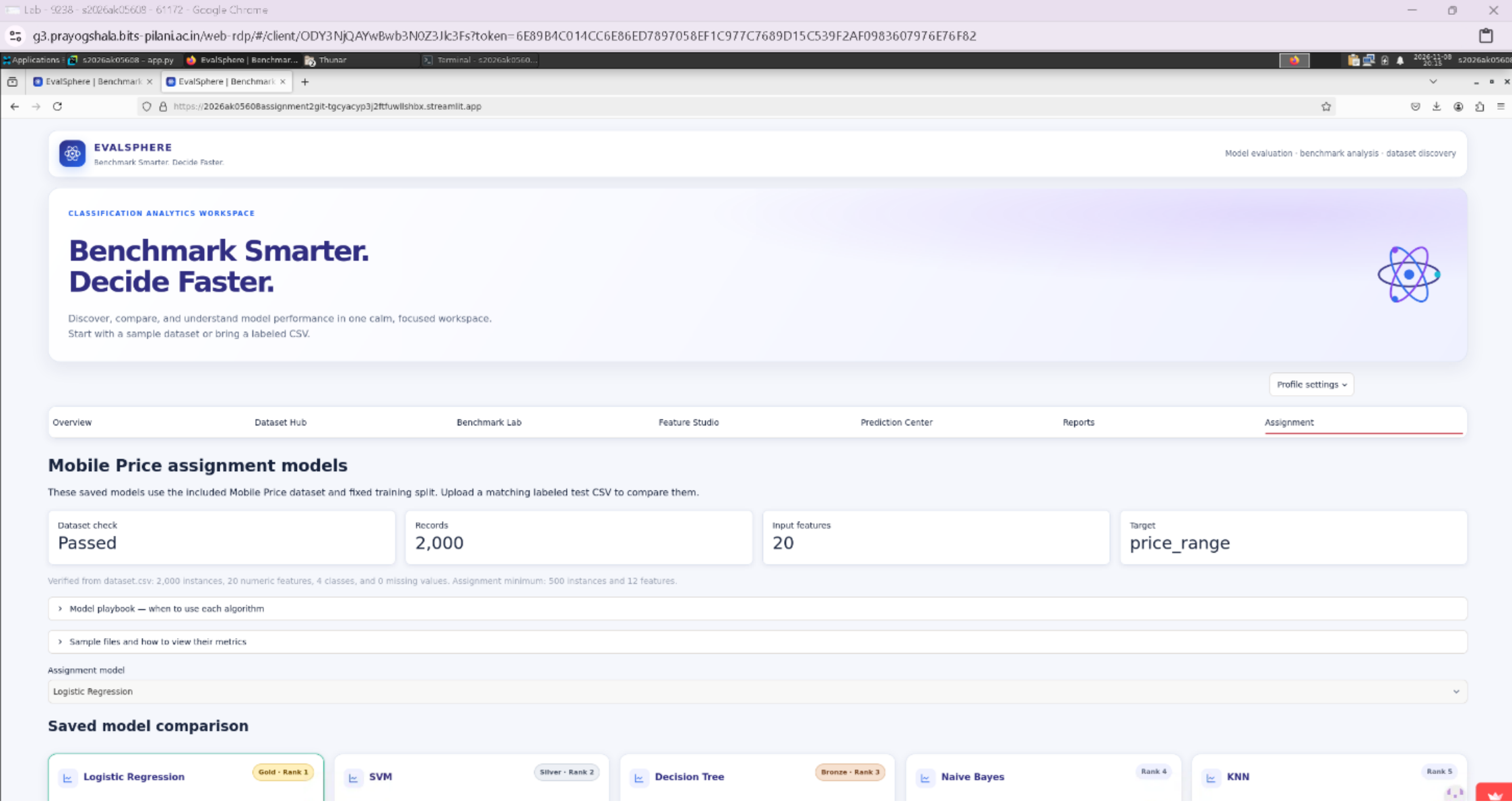
02 Streamlit application launch

The lab terminal showing the Streamlit server running and the local application URL.



03 EvalSphere assignment workspace

The application displaying the Mobile Price assignment and saved-model comparison workflow.



04 Live Streamlit app in the BITS Virtual Lab

The deployed EvalSphere app open in the lab browser, showing the assignment results workspace.

EvalSphere — ML Model Comparison and Be

Benchmark Smarter. Decide Faster.

Student: SRIRAM POLAMARASETTI BITS ID: 2026ak05608

a. Problem statement

The objective of this assignment is to build, train, and evaluate five machine learning classification models on a dataset that satisfies the assignment requirements. The models are compared using Accuracy, AUC, Precision, Recall, F1, and Matthews Correlation Coefficient (MCC).

EvalSphere presents the trained models and their evaluation in an interactive Streamlit application. Users can inspect the saved-model comparison, upload a labeled test CSV, select a model, review evaluation metrics, and view a confusion matrix and classification report. Uploaded test data is evaluated with the saved models and is not used to retrain them.

b. Dataset description

Property	Description
Dataset	Mobile Price Classification
Problem type	Multiclass classification
Number of records	2,000
Number of input features	20 numeric features
Target column	price_range
Target classes	0, 1, 2, and 3
Records per class	500
Missing values	None

The dataset has four price-range classes. Its 20 input features describe mobile phone specifications, including battery power, clock speed, memory, camera specifications, screen dimensions, and connectivity. With 2,000 records and 20 input features, it exceeds the minimum requirement of 500 records and 12 features.

The source dataset is dataset.csv (./dataset.csv). A stratified 80/20 split with random seed 42 produces 1,600 training records and a 400-record holdout. The scaler is fitted on the training data only. The project also includes a balanced 100-record sample in sample_test_data.csv (./sample_test_data.csv) and the complete holdout in test_data.csv (./test_data.csv).

c. GitHub Repository Link

Repository URL: EvalSphere Assignment 2 on GitHub (https://github.com/sreeram348/2026ak05608_assignment_2.git)

The repository contains the application source, model training and evaluation code, dataset and test CSV files, saved model artifacts, requirements, and this README.

d. Models used and comparison

The five models use the same stratified training and holdout split. Input features are standardized using a scaler fitted on the training partition. Precision, Recall, and F1 are weighted averages. Multiclass AUC is calculated using weighted one-vs-rest.

Evaluation metrics

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9650	0.9987	0.9650	0.9650	0.9650	0.9534
Decision Tree	0.8300	0.8867	0.8319	0.8300	0.8302	0.7738
KNN	0.5000	0.7697	0.5211	0.5000	0.5054	0.3350
Naive Bayes	0.8100	0.9506	0.8113	0.8100	0.8105	0.7468
Support Vector Machine	0.8950	0.9580	0.8969	0.8950	0.8956	0.8602

Observations on model performance

ML Model Name	Observation about model performance
Logistic Regression	Observation about model performance: Best overall result on this holdout: 96.50% accuracy, 0.9987 AUC, 0.9650 F1, and 0.9534 MCC. It provides the strongest overall classification performance among the five evaluated models.
Decision Tree	Observation about model performance: Achieved 83.00% accuracy. Its lower 0.8867 AUC and 0.7738 MCC show weaker performance on this holdout than Logistic Regression and SVM.
KNN	Observation about model performance: Achieved 50.00% accuracy, the lowest result in this comparison. With five neighbors and standardized features, this tested configuration was the weakest on the selected holdout.
Naive Bayes	Observation about model performance: Achieved 81.00% accuracy and 0.9506 AUC. Its AUC is higher than the Decision Tree's, although its accuracy and MCC are lower.
Support Vector Machine	Observation about model performance: Achieved 89.50% accuracy, 0.9580 AUC, and 0.8602 MCC. It ranked second to Logistic Regression on accuracy and MCC in this comparison.

Overall winner for this dataset

Logistic Regression is the overall winner on the fixed 400-record holdout. It achieved the highest Accuracy, AUC, Precision, Recall, F1, and MCC among the five models. This conclusion applies to this dataset split and is not a guarantee of performance on other data.

Step 6. Deploy on Streamlit Community Cloud

Live application

Streamlit app URL: EvalSphere | Benchmark Smarter. Decide Faster. (https://2026ak05608assignment2git-tgcyacyp3j2ftfuwllshbx.streamlit.app/)

Deployment steps

1. Open Streamlit Community Cloud (https://streamlit.io/cloud) and sign in with the GitHub account that can access the repository. 2. Select New app. 3. Choose the EvalSphere GitHub repository and the deployment branch (usually main). 4. Set the application file to app.py. 5. Confirm the Python dependencies are listed in requirements.txt. 6. Select Deploy and wait for the application health check to complete. 7. Open the deployed app URL above to use EvalSphere.

Required application features

Requirement	EvalSphere implementation
Dataset upload option (CSV)	EvalSphere implementation: The Assignment workspace accepts a labeled CSV test file. Upload test data only; the app does not train a model on uploaded test records.
Model selection dropdown	EvalSphere implementation: Select one of the five saved models in the Assignment workspace.
Evaluation metrics	EvalSphere implementation: The app calculates Accuracy, AUC, Precision, Recall, F1, and MCC on the uploaded labeled test data and displays the saved-model comparison.
Confusion matrix or classification report	EvalSphere implementation: The selected model's confusion matrix and per-class classification report are displayed with labeled results.
Saved-model evaluation without retraining	EvalSphere implementation: The app loads the saved models and scaler, validates the upload, transforms its features, and evaluates predictions without fitting or retraining on the uploaded test data.
Results for different models	EvalSphere implementation: The uploaded test-data comparison reports results for all five saved models on the same rows.

Test CSV requirements

The uploaded CSV must contain a header row, all 20 Mobile Price input feature columns, and the price_range target column. The app validates the required columns and target labels before evaluation. The included sample_test_data.csv (./sample_test_data.csv) is a balanced 100-row file for a quick test. Use test_data.csv (./test_data.csv) for the full 400-row holdout evaluation. Upload limits are enforced by the application.

Run the project locally

Use Python 3.11 or newer:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python train_models.py
streamlit run app.py
```

Running train_models.py trains and saves the five models and scaler, writes the reference and sample test CSV files, and updates results.csv.